

C. Abdul Hakeem College of Engineering & Technology
Department of Master of Computer Applications
MC4301 - Machine Learning
Unit - 2
Model Evaluation and Feature Engineering

Model Selection

The challenge of applied machine learning, therefore, becomes how to choose among a range of different models that you can use for your problem.

Naively, you might believe that model performance is sufficient, but should you consider other concerns, such as how long the model takes to train or how easy it is to explain to project stakeholders. Their concerns become more pressing if a chosen model must be used operationally for months or years.

What Is Model Selection

Model selection is the process of selecting one final machine learning model from among a collection of candidate machine learning models for a training dataset.

Model selection is a process that can be applied both across different types of models (e.g. logistic regression, SVM, KNN, etc.) and across models of the same type configured with different model hyperparameters (e.g. different kernels in an SVM).

When we have a variety of models of different complexity (e.g., linear or logistic regression models with different degree polynomials, or KNN classifiers with different values of K), how should we pick the right one?

For example, we may have a dataset for which we are interested in developing a classification or regression predictive model. We do not know beforehand as to which model will perform best on this problem, as it is unknowable. Therefore, we fit and evaluate a suite of different models on the problem.

Model selection is the process of choosing one of the models as the final model that addresses the problem.

Model selection is different from **model assessment**.

For example, we evaluate or assess candidate models in order to choose the best one, and this is model selection. Whereas once a model is chosen, it can be evaluated in order to communicate how well it is expected to perform in general; this is model assessment.

The process of evaluating a model's performance is known as model assessment, whereas the process of selecting the proper level of flexibility for a model is known as model selection.

Considerations for Model Selection

Fitting models is relatively straightforward, although selecting among them is the true challenge of applied machine learning.

Firstly, we need to get over the idea of a “*best*” model.

All models have some predictive error, given the statistical noise in the data, the incompleteness of the data sample, and the limitations of each different model type. Therefore, the notion of a perfect or best model is not useful. Instead, we must seek a model that is “*good enough*.”

What do we care about when choosing a final model?

The project stakeholders may have specific requirements, such as maintainability and limited model complexity. As such, a model that has lower skill but is simpler and easier to understand may be preferred.

Alternately, if model skill is prized above all other concerns, then the ability of the model to perform well on out-of-sample data will be preferred regardless of the computational complexity involved.

Therefore, a “*good enough*” model may refer to many things and is specific to your project, such as:

- A model that meets the requirements and constraints of project stakeholders.
- A model that is sufficiently skillful given the time and resources available.
- A model that is skillful as compared to naive models.
- A model that is skillful relative to other tested models.
- A model that is skillful relative to the state-of-the-art.

Next, we must consider what is being selected.

For example, we are not selecting a fit model, as all models will be discarded. This is because once we choose a model, we will fit a new final model on all available data and start using it to make predictions.

Therefore, are we choosing among algorithms used to fit the models on the training dataset?

Some algorithms require specialized data preparation in order to best expose the structure of the problem to the learning algorithm. Therefore, we must go one step further and consider **model selection as the process of selecting among model development pipelines**.

Each pipeline may take in the same raw training dataset and outputs a model that can be evaluated in the same manner but may require different or overlapping computational steps, such as:

- Data filtering.

- Data transformation.
- Feature selection.
- Feature engineering.
- And more...

Model Selection Techniques

The best approach to model selection requires “*sufficient*” data, which may be nearly infinite depending on the complexity of the problem.

In this ideal situation, we would split the data into training, validation, and test sets, then fit candidate models on the training set, evaluate and select them on the validation set, and report the performance of the final model on the test set.

If we are in a data-rich situation, the best approach [...] is to randomly divide the dataset into three parts: a training set, a validation set, and a test set. The training set is used to fit the models; the validation set is used to estimate prediction error for model selection; the test set is used for assessment of the generalization error of the final chosen model.

This is impractical on most predictive modeling problems given that we rarely have sufficient data, or are able to even judge what would be sufficient.

In many applications, however, the supply of data for training and testing will be limited, and in order to build good models, we wish to use as much of the available data as possible for training. However, if the validation set is small, it will give a relatively noisy estimate of predictive performance.

Instead, there are two main classes of techniques to approximate the ideal case of model selection; they are:

- **Probabilistic Measures:** Choose a model via in-sample error and complexity.
- **Resampling Methods:** Choose a model via estimated out-of-sample error.

Probabilistic Measures

Probabilistic measures involve analytically scoring a candidate model using both its performance on the training dataset and the complexity of the model.

It is known that training error is optimistically biased, and therefore is not a good basis for choosing a model. The performance can be penalized based on how optimistic the training error is believed to be. This is typically achieved using algorithm-specific methods, often linear, that penalize the score based on the complexity of the model.

Historically various ‘information criteria’ have been proposed that attempt to correct for the bias of maximum likelihood by the addition

of a penalty term to compensate for the over-fitting of more complex models.

A model with fewer parameters is less complex, and because of this, is preferred because it is likely to generalize better on average.

Four commonly used probabilistic model selection measures include:

- Akaike Information Criterion (AIC).
- Bayesian Information Criterion (BIC).
- Minimum Description Length (MDL).
- Structural Risk Minimization (SRM).

Probabilistic measures are appropriate when using simpler linear models like linear regression or logistic regression where the calculating of model complexity penalty (e.g. in sample bias) is known and tractable.

Resampling Methods

Resampling methods seek to estimate the performance of a model (or more precisely, the model development process) on out-of-sample data.

This is achieved by splitting the training dataset into sub train and test sets, fitting a model on the sub train set, and evaluating it on the test set. This process may then be repeated multiple times and the mean performance across each trial is reported.

It is a type of Monte Carlo estimate of model performance on out-of-sample data, although each trial is not strictly independent as depending on the resampling method chosen, the same data may appear multiple times in different training datasets, or test datasets.

Three common resampling model selection methods include:

- Random train/test splits.
- Cross-Validation (k-fold, LOOCV, etc.).
- Bootstrap.

Most of the time probabilistic measures are not available, therefore resampling methods are used.

By far the most popular is the cross-validation family of methods that includes many subtypes.

Probably the simplest and most widely used method for estimating prediction error is cross-validation.

An example is the widely used k-fold cross-validation that splits the training dataset into k folds where each example appears in a test set only once.

Another is the leave one out (LOOCV) where the test set is comprised of a single sample and each sample is given an opportunity to be the test set, requiring N (the number of samples in the training set) models to be constructed and evaluated.

Training Model

A machine learning training model is a process in which a machine learning (ML) algorithm is fed with sufficient training data to learn from.

ML models can be trained to benefit manufacturing processes in several ways. The ability of ML models to process large volumes of data can help manufacturers identify anomalies and test correlations while searching for patterns across the data feed. It can equip manufacturers with predictive maintenance capabilities and minimize planned and unplanned downtime.

A training model is a dataset that is used to train an ML algorithm. It consists of the sample output data and the corresponding sets of input data that have an influence on the output. The training model is used to run the input data through the algorithm to correlate the processed output against the sample output. The result from this correlation is used to modify the model.

This iterative process is called “model fitting”. The accuracy of the training dataset or the validation dataset is critical for the precision of the model.

Model training in machine language is the process of feeding an ML algorithm with data to help identify and learn good values for all attributes involved. There are several types of machine learning models, of which the most common ones are supervised and unsupervised learning.

Supervised learning is possible when the training data contains both the input and output values. Each set of data that has the inputs and the expected output is called a supervisory signal. The training is done based on the deviation of the processed result from the documented result when the inputs are fed into the model.

Unsupervised learning involves determining patterns in the data. Additional data is then used to fit patterns or clusters. This is also an iterative process that improves the accuracy based on the correlation to the expected patterns or clusters. There is no reference output dataset in this method.

The process of training an ML model involves providing an ML algorithm (that is, the *learning algorithm*) with training data to learn from. The term *ML model* refers to the model artifact that is created by the training process.

The training data must contain the correct answer, which is known as a *target* or *target attribute*. The learning algorithm finds patterns in the training data that map the input data attributes to the target (the answer that you want to predict), and it outputs an ML model that captures these patterns.

You can use the ML model to get predictions on new data for which you do not know the target. For example, let's say that you want to train an ML model to predict if an email is spam or not spam. You would provide Amazon ML with training data that contains emails for which you know the target (that is, a label that tells whether an email is spam or not spam). Amazon ML would train an ML model by using this data, resulting in a model that attempts to predict whether new email will be spam or not spam.

Types of ML Models

Amazon ML supports three types of ML models: binary classification, multiclass classification, and regression. The type of model you should choose depends on the type of target that you want to predict.

Binary Classification Model

ML models for binary classification problems predict a binary outcome (one of two possible classes). To train binary classification models, Amazon ML uses the industry-standard learning algorithm known as logistic regression.

Examples of Binary Classification Problems

"Is this email spam or not spam?"

"Will the customer buy this product?"

"Is this product a book or a farm animal?"

"Is this review written by a customer or a robot?"

Multiclass Classification Model

ML models for multiclass classification problems allow you to generate predictions for multiple classes (predict one of more than two outcomes). For training multiclass models, Amazon ML uses the industry-standard learning algorithm known as multinomial logistic regression.

Examples of Multiclass Problems

"Is this product a book, movie, or clothing?"

"Is this movie a romantic comedy, documentary, or thriller?"

"Which category of products is most interesting to this customer?"

Regression Model

ML models for regression problems predict a numeric value. For training regression models, Amazon ML uses the industry-standard learning algorithm known as linear regression.

Examples of Regression Problems

"What will the temperature be in Seattle tomorrow?"

"For this product, how many units will sell?"

"What price will this house sell for?"

Training Process

To train an ML model, you need to specify the following:

Input training datasource

Name of the data attribute that contains the target to be predicted

Required data transformation instructions

Training parameters to control the learning algorithm

Training Parameters

Typically, machine learning algorithms accept parameters that can be used to control certain properties of the training process and of the resulting ML model. In Amazon Machine Learning, these are called *training parameters*. You can set these parameters using the Amazon ML console, API, or command line interface (CLI). If you do not set any parameters, Amazon ML will use default values that are known to work well for a large range of machine learning tasks.

You can specify values for the following training parameters:

Maximum model size

Maximum number of passes over training data

Shuffle type

Regularization type

Regularization amount

Maximum Model Size

The maximum model size is the total size, in units of bytes, of patterns that Amazon ML creates during the training of an ML model.

By default, Amazon ML creates a 100 MB model. You can instruct Amazon ML to create a smaller or larger model by specifying a different size. For the range of available sizes, see [Types of ML Models](#)

If Amazon ML can't find enough patterns to fill the model size, it creates a smaller model. For example, if you specify a maximum model size of 100 MB, but Amazon ML finds patterns that total only 50 MB, the resulting model will be 50 MB. If Amazon ML finds more patterns than will fit into the specified size, it enforces a maximum cut-off by trimming the patterns that least affect the quality of the learned model.

Choosing the model size allows you to control the trade-off between a model's predictive quality and the cost of use. Smaller models can cause Amazon ML to remove many patterns to fit within the maximum size limit, affecting the quality of predictions. Larger models, on the other hand, cost more to query for real-time predictions.

Maximum Number of Passes over the Data

For best results, Amazon ML may need to make multiple passes over your data to discover patterns. By default, Amazon ML makes 10 passes, but you can change the default by setting a number up to 100. Amazon ML keeps track of the quality of patterns (model convergence) as it goes along, and automatically stops the training when there are no more data points or patterns to discover. For example, if you set the number of passes to 20, but Amazon ML discovers that no new patterns can be found by the end of 15 passes, then it will stop the training at 15 passes.

Shuffle Type for Training Data

In Amazon ML, you must shuffle your training data. Shuffling mixes up the order of your data so that the SGD algorithm doesn't encounter one type of data for too many observations in succession. For example, if you are training an ML model to predict a product type, and your training data includes movie, toy, and video game product types, if you sorted the data by the product type column before uploading it, the algorithm sees the data alphabetically by product type. The algorithm sees all of your data for movies first, and your ML model begins to learn patterns for movies. Then, when your model encounters data on toys, every update that the algorithm makes would fit the model to the toy product type, even if those updates degrade the patterns that fit movies. This sudden switch from movie to toy type can produce a model that doesn't learn how to predict product types accurately.

Regularization Type and Amount

The predictive performance of complex ML models (those having many input attributes) suffers when the data contains too many patterns. As the number of patterns increases, so does the likelihood that the model learns unintentional data artifacts, rather than true data patterns. In such a case, the model does very well on the training data, but can't generalize well on new data. This phenomenon is known as *overfitting* the training data.

Regularization helps prevent linear models from overfitting training data examples by penalizing extreme weight values. L1 regularization reduces the number of features used in the model by pushing the weight of features that would otherwise have very small weights to zero. L1 regularization produces sparse models and reduces the

amount of noise in the model. L2 regularization results in smaller overall weight values, which stabilizes the weights when there is high correlation between the features. You can control the amount of L1 or L2 regularization by using the Regularization amount parameter. Specifying an extremely large Regularization amount value can cause all features to have zero weight.

Selecting and tuning the optimal regularization value is an active subject in machine learning research. You will probably benefit from selecting a moderate amount of L2 regularization, which is the default in the Amazon ML console. Advanced users can choose between three types of regularization (none, L1, or L2) and amount. For more information about regularization, go to Regularization (mathematics)

Training Parameters: Types and Default Values

Following table lists the Amazon ML training parameters, along with the default values and the allowable range for each.

Training Parameter	Type	Default Value	Description
maxMLModelSizeInBytes	Integer	100,000,000 bytes (100 MiB)	Allowable range: 100,000 (100 KiB) to 2,147,483,648 (2 GiB) Depending on the input data, the model size might affect the performance.
sgd.maxPasses	Integer	10	Allowable range: 1-100
sgd.shuffleType	String	auto	Allowable values: auto or none Allowable range: 0 to MAX_DOUBLE
sgd.l1RegularizationAmount	Double	0 (By default, L1 isn't used)	L1 values between 1E-4 and 1E-8 have been found to produce good results. Larger values are likely to produce models that aren't very useful.
sgd.l2RegularizationAmount	Double	1E-6 (By default, L2 is used with this amount of regularization)	You can't set both L1 and L2. You must choose one or the other. Allowable range: 0 to MAX_DOUBLE L2 values between 1E-2 and 1E-6 have been found to produce good results. Larger values are likely to produce models that aren't very

Training Parameter	Type	Default Value	Description
			useful.
			You can't set both L1 and L2. You must choose one or the other.

Creating an ML Model

After you've created a datasource, you are ready to create an ML model. If you use the Amazon Machine Learning console to create a model, you can choose to use the default settings or you customize your model by applying custom options.

Custom options include:

Evaluation settings: You can choose to have Amazon ML reserve a portion of the input data to evaluate the predictive quality of the ML model.

A recipe: A recipe tells Amazon ML which attributes and attribute transformations are available for model training.

Training parameters: Parameters control certain properties of the training process and of the resulting ML model.

Creating an ML Model with Default Options

Choose the **Default** options, if you want Amazon ML to:

Split the input data to use the first 70 percent for training and use the remaining 30 percent for evaluation

Suggest a recipe based on statistics collected on the training datasource, which is 70 percent of the input datasource

Choose default training parameters

Creating an ML Model with Custom Options

Customizing your ML model allows you to:

Provide your own recipe.

Choose training parameters.

Choose a training/evaluation splitting ratio other than the default 70/30 ratio or provide another datasource that you have already prepared for evaluation.

You can also choose the default values for any of these settings.

If you've already created a model using the default options and want to improve your model's predictive performance, use the **Custom** option to create a new model with some customized settings. For example, you might add more feature transformations to the recipe or increase the number of passes in the training parameter.

Model Representation and Interpretability

Machine Learning Models

What is a machine learning Model?

A machine learning model is a program that can find patterns or make decisions from a previously unseen dataset. For example, in natural language processing, machine learning models can parse and correctly recognize the intent behind previously unheard sentences or combinations of words. In image recognition, a machine learning model can be taught to recognize objects - such as cars or dogs. A machine learning model can perform such tasks by having it 'trained' with a large dataset. During training, the machine learning algorithm is optimized to find certain patterns or outputs from the dataset, depending on the task. The output of this process - often a computer program with specific rules and data structures - is called a machine learning model.

What is a machine learning Algorithm?

A machine learning algorithm is a mathematical method to find patterns in a set of data. Machine Learning algorithms are often drawn from statistics, calculus, and linear algebra. Some popular examples of machine learning algorithms include linear regression, decision trees, random forest, and XGBoost.

What is Model Training in machine learning?

The process of running a machine learning algorithm on a dataset (called training data) and optimizing the algorithm to find certain patterns or outputs is called model training. The resulting function with rules and data structures is called the trained machine learning model.

What are the different types of Machine Learning?

In general, most machine learning techniques can be classified into supervised learning, unsupervised learning, and reinforcement learning.

What is Supervised Machine Learning?

In supervised machine learning, the algorithm is provided an input dataset, and is rewarded or optimized to meet a set of specific outputs. For example, supervised machine learning is widely deployed in image recognition, utilizing a technique called classification. Supervised machine learning is also used in predicting demographics such as population growth or health metrics, utilizing a technique called regression.

What is Unsupervised Machine Learning?

In unsupervised machine learning, the algorithm is provided an input dataset, but not rewarded or optimized to specific outputs, and instead trained to group objects by common characteristics. For example, recommendation engines on online stores rely on unsupervised machine learning, specifically a technique called clustering.

What is Reinforcement Learning?

In reinforcement learning, the algorithm is made to train itself using many trial and error experiments. Reinforcement learning happens when the algorithm interacts continually with the environment, rather than relying on training data. One of the most popular examples of reinforcement learning is autonomous driving.

What are the different machine learning models?

There are many machine learning models, and almost all of them are based on certain machine learning algorithms. Popular classification and regression algorithms fall under supervised machine learning, and clustering algorithms are generally deployed in unsupervised machine learning scenarios.

Supervised Machine Learning

- Logistic Regression: Logistic Regression is used to determine if an input belongs to a certain group or not
- SVM: SVM, or Support Vector Machines create coordinates for each object in an n-dimensional space and uses a hyperplane to group objects by common features
- Naive Bayes: Naive Bayes is an algorithm that assumes independence among variables and uses probability to classify objects based on features
- Decision Trees: Decision trees are also classifiers that are used to determine what category an input falls into by traversing the leaf's and nodes of a tree
- Linear Regression: Linear regression is used to identify relationships between the variable of interest and the inputs, and predict its values based on the values of the input variables.
- kNN: The k Nearest Neighbors technique involves grouping the closest objects in a dataset and finding the most frequent or average characteristics among the objects.
- Random Forest: Random forest is a collection of many decision trees from random subsets of the data, resulting in a combination of trees that may be more accurate in prediction than a single decision tree.
- Boosting algorithms: Boosting algorithms, such as Gradient Boosting Machine, XGBoost, and LightGBM, use ensemble learning. They combine the predictions from multiple algorithms (such as decision trees) while taking into account the error from the previous algorithm.

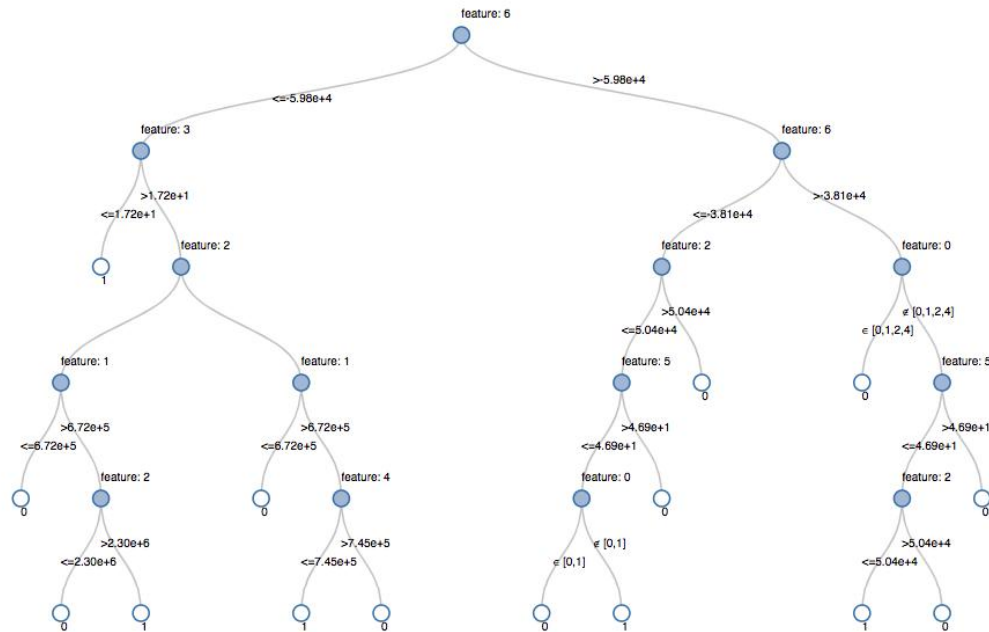
Unsupervised Machine Learning

- K-Means: The K-Means algorithm finds similarities between objects and groups them into K different clusters.

- Hierarchical Clustering: Hierarchical clustering builds a tree of nested clusters without having to specify the number of clusters.

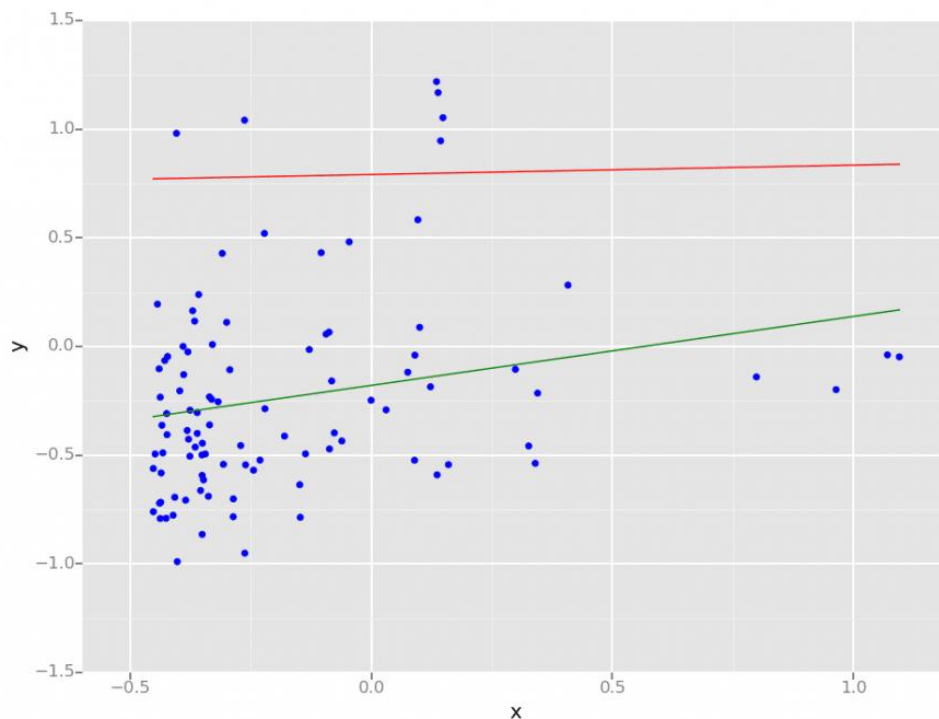
What is a Decision Tree in Machine Learning (ML)?

A Decision Tree is a predictive approach in ML to determine what class an object belongs to. As the name suggests, a decision tree is a tree-like flow chart where the class of an object is determined step-by-step using certain known conditions.



What is Regression in Machine Learning?

Regression in data science and machine learning is a statistical method that enables predicting outcomes based on a set of input variables. The outcome is often a variable that depends on a combination of the input variables.



What is a Classifier in Machine Learning?

A classifier is a machine learning algorithm that assigns an object as a member of a category or group. For example, classifiers are used to detect if an email is spam, or if a transaction is fraudulent.

How many models are there in machine learning?

Many! Machine learning is an evolving field and there are always more machine learning models being developed.

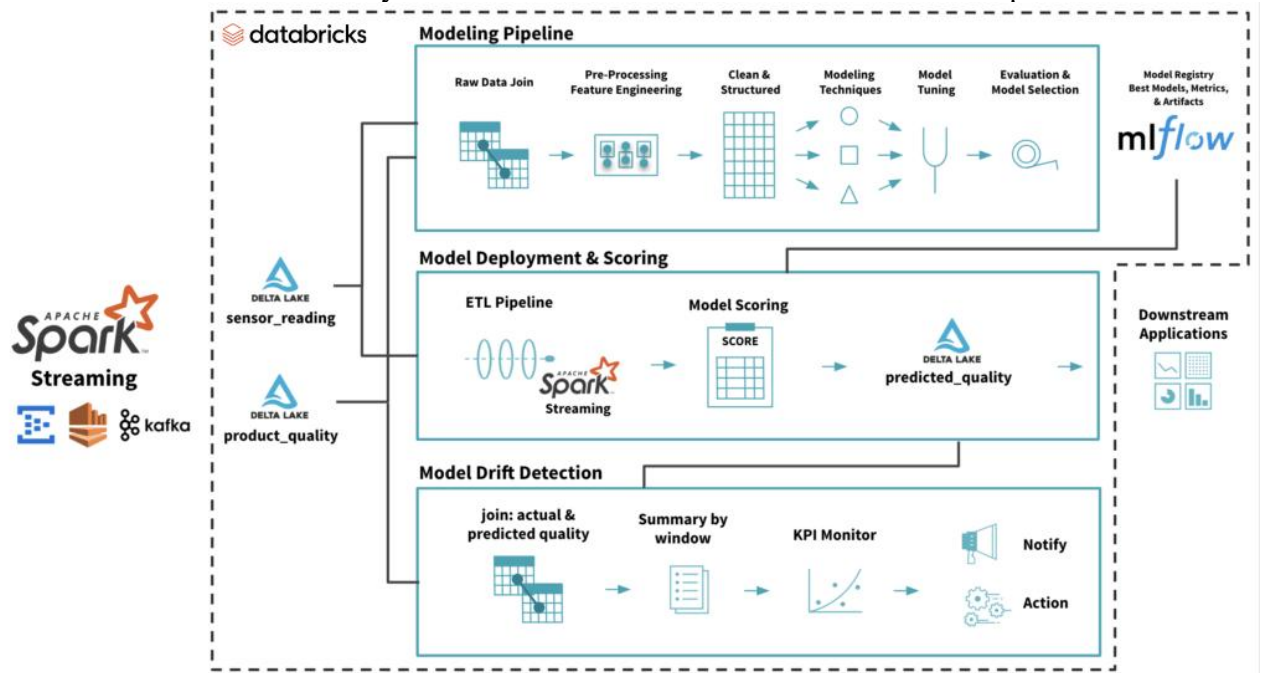
What is the best model for machine learning?

The machine learning model most suited for a specific situation depends on the desired outcome. For example, to predict the number of vehicle purchases in a city from historical data, a supervised learning technique such as linear regression might be most useful. On the other hand, to identify if a potential customer in that city would purchase a vehicle, given their income and commuting history, a decision tree might work best.

What is model deployment in Machine Learning (ML)?

Model deployment is the process of making a machine learning model available for use on a target environment—for testing or production. The model is usually integrated with other applications in the environment (such as databases and UI) through APIs. Deployment is the stage after which an organization can actually make

a return on the heavy investment made in model development.



Model Interpretability

What is Model Interpretability in Machine Learning?

A machine learning algorithm's interpretability refers to how easy it is for humans to understand the processes it uses to arrive at its outcomes. Until recently, artificial intelligence (AI) algorithms have been notorious for being "black boxes," providing no way to understand their inner processes and making it difficult to explain resulting insights to regulatory agencies and stakeholders.

Some models, like logistic regression, are considered to be fairly straightforward and therefore highly interpretable, but as you add features or use more complicated machine learning models such as deep learning, interpretability gets more and more difficult.

Why is Model Interpretability Important?

When using an algorithm's outcomes to make high-stakes decisions, it's important to know which features the model did and did not take into account. Additionally, if a model isn't highly interpretable, the business might not be legally permitted to use its insights to make changes to processes. In heavily regulated industries like banking, insurance, and healthcare, it is important to be able to understand the factors that contribute to likely outcomes in order to comply with regulation and industry best practices.

Evaluating Performance of a Model

We expect machine learning models to provide accurate and trustworthy predictions. To confidently trust their predictions, it is important to assess how machine learning models generalize on test data. Let us look at how to test model performance.

Training set – according to this insightful article on model evaluation, this refers to a subset of a dataset used to build predictive models. It includes a set of input examples that will be used to train a model by adjusting the parameters of the set.

Validation set – is a subset of a dataset whose purpose is to assess the performance of the model built, during the training phase. It periodically evaluates a model and allows for fine-tuning of the parameters of the model. This post mentions that not all modeling algorithms need a validation set.

Test set – this is also known as unseen data. It is the final evaluation that a model undergoes after the training phase. A test set is best defined in this article as a subset of a dataset used to assess the possible future performance of a model. For example, if a model fits the training better than the test set, overfitting is likely present.

Overfitting– refers to when a model contains more parameters than can be accounted for by the dataset. Noisy data contributes to overfitting. The generalization of these models is unreliable since the model learns more than it is meant to from the dataset.

Why evaluate performance?

Machine learning has become integral to our daily lives. We interact with some form of machine learning every single day. Since we truly depend on machine learning models for various reasons, it's important to have models that provide accurate and trustworthy predictions for their respective use cases. We must always test how a model generalizes on unseen data.

For example, in an enterprise setting, these models need to offer real value to the business by producing the highest levels of performance. But how do we evaluate the performance of a model? For classification problems, a very common and obvious answer is to measure its accuracy.

Model evaluation techniques

The techniques to evaluate the performance of a model can be divided into two parts: cross-validation and holdout. Both these techniques make use of a test set to assess model performance.

Cross validation

Cross-validation involves the use of a training dataset and an independent dataset. These two sets result from partitioning the original dataset. The sets are used to evaluate an algorithm.

Let's explore how.

First, we split the dataset into groups of instances equal in size. These groups are called folds. The model to be evaluated is trained on all the folds except one. After training, we test the model on the fold that was excluded. This process is then repeated over and over again, depending on the number of folds.

If there are six folds, we will repeat the process six times. The reason for the repetition is that each fold gets to be excluded and act as the test set. Last, we measure the average performance across all folds to get an estimation of how effective the algorithm is on a problem.

A popular cross-validation technique is the k-fold cross-validation. It uses the same steps described above. The k, (is a user-specified number), stands for the number of folds. The value of k may vary based on the size of the dataset but as an example, let us use a scenario of 4-fold cross-validation.

The model will be trained and tested four times. Let's say the first-round trains on folds 1,2 and 3. The testing will be on fold 4. For the second round, it may train on folds 1,2, and 4 and test on fold 3. For the third, it may train on folds 1,3, and 4 and test on fold 2.

The last round will test on folds 2,3 and 4 and test on fold 1. The interchange between training and test data makes this method very effective. However, compared to the holdout technique, cross-validation takes more time to run and uses more computational resources.

Holdout

It's important to get an unbiased estimate of model performance. This is exactly what the holdout technique offers. To get this unbiased estimate, we test a model on data different from the data we trained it on. This technique divides a dataset into three subsets: training, validation, and test sets.

From the terms we defined at the start of the article, we know that the training set helps the model make predictions and that the test set assesses the performance of the model. The validation set also helps to assess the performance of the model by providing an environment to fine-tune the parameters of the model. From this, we select the best performing model.

The holdout method is ideal when dealing with a very large dataset, it prevents model overfitting, and incurs lower computational costs.

When a function fits too tightly to a set of data points, an error known as overfitting occurs. As a result, a model performs poorly on unseen data. To detect overfitting, we could first split our dataset into training and test sets. We then monitor the performance of the model on both training data and test data.

If our model offers superior performance on the training set when compared to the test set, there's a good chance overfitting is present. For instance, a model might offer 90% accuracy on the training set yet give 50% on the test set.

Model evaluation metrics

Metrics for classification problems

Predictions for classification problems yield four types of outcomes: true positives, true negatives, false positives, and false negatives. We'll define them later on. We look at a few metrics for classification problems.

Classification accuracy

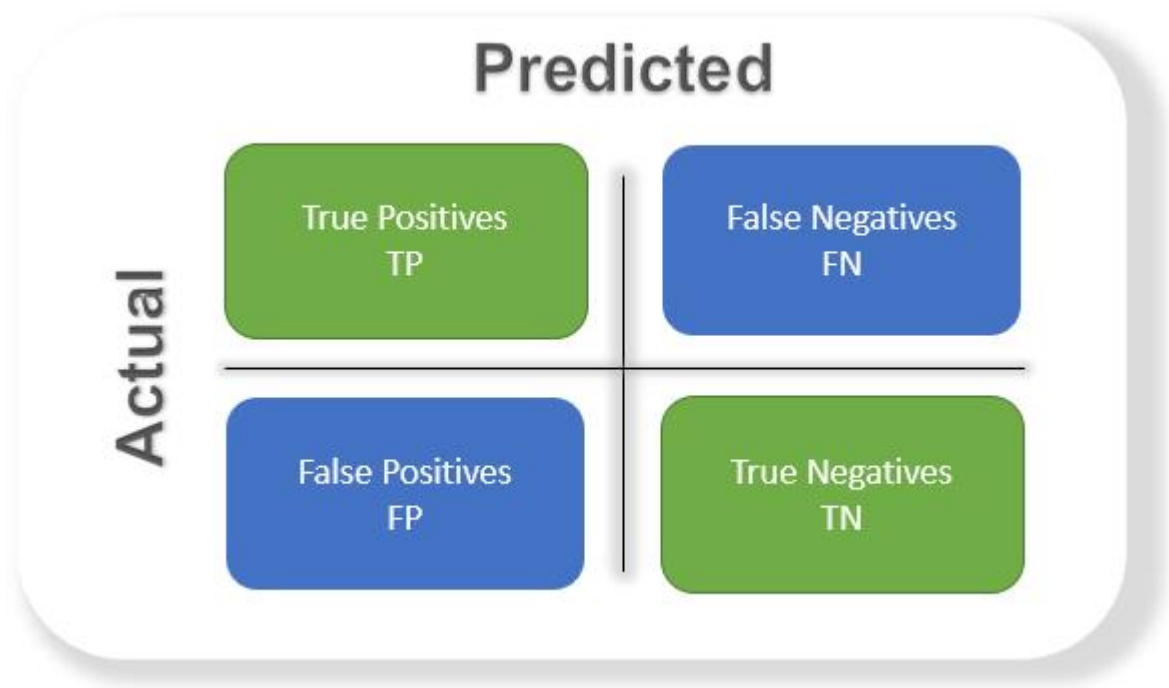
The most common evaluation metric for classification problems is accuracy. It's taken as the number of correct predictions against the total number of predictions made (or input samples). However, as much as accuracy is used to evaluate a model, it's not a clear indicator of model performance as we stated earlier.

Classification accuracy works best if the samples belonging to each class are equal in number. Consider a scenario with 97% samples from class X and 3% from class Y in a training set. A model can very easily achieve 97% training accuracy by predicting each training sample in class X.

Testing the same model on a test set with 55% samples of X and 45% samples of Y, the test accuracy is reduced to 55%. This is why classification accuracy is not a clear indicator of performance. It provides a false sense of attaining high levels of accuracy.

Confusion matrix

The confusion matrix forms the basis for the other types of classification metrics. It's a matrix that fully describes the performance of the model. A confusion matrix gives an in-depth breakdown of the correct and incorrect classifications of each class.



Confusion Matrix

The four terms represented in the image above are very important.

Let's define them:

True positives – a scenario where positive predictions are actually positive.

True negatives – negative predictions are actually negative.

False positives – positive predictions are actually negative.

False negatives – a scenario where negative predictions are actually positive.

From the definition of the four terms above, the takeaway is that it's important to amplify true positives and true negatives. False positives and false negatives represent misclassification, that could be costly in real-world applications. Consider instances of misdiagnosis in a medical deployment of a model.

A model may wrongly predict that a healthy person has cancer. It may also classify someone who actually has cancer as cancer-free. Both these outcomes would have unpleasant consequences in terms of the well being of the patients after being diagnosed (or finding out about the misdiagnosis), treatment plans as well as expenses. Therefore it's important to minimize false negatives and false positives.

The green shapes in the image represent when the model makes the correct prediction. The blue ones represent scenarios where the model made the wrong predictions. The

rows of the matrix represent the actual classes while the columns represent predicted classes.

We can calculate accuracy from the confusion matrix. The accuracy is given by taking the average of the values in the “true” diagonal.

That is: $\text{Accuracy} = (\text{True Positive} + \text{True Negative}) / \text{Total Sample}$

That translates to: $\text{Accuracy} = \text{Total Number of Correct Predictions} / \text{Total Number of Observations}$

Since the confusion matrix visualizes the four possible outcomes of classification mentioned above, aside from accuracy, we have insight into precision, recall, and ultimately, F-score. They can easily be calculated from the matrix. Precision, recall, and F-score are defined in the section below.

F-score

F-score is a metric that incorporates both the precision and recall of a test to determine the score. This post defines it as the harmonic mean of recall and precision. F-score is also known as F-measure or F1 score.

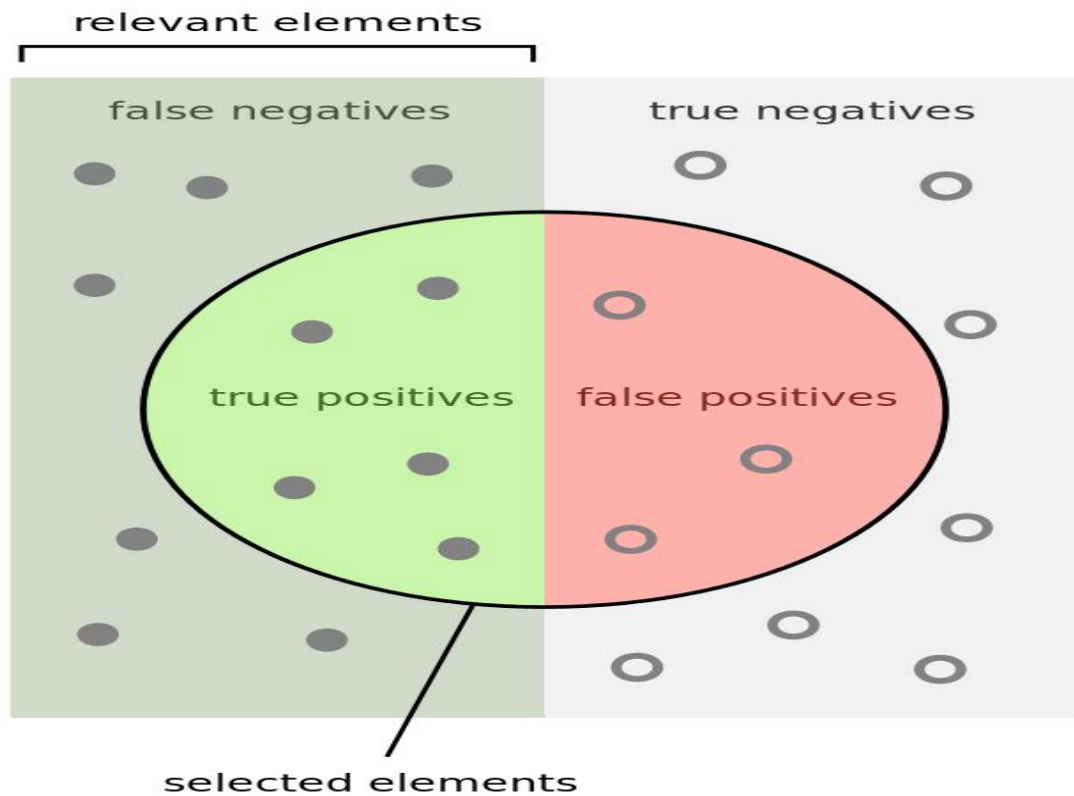
Let's define precision and recall.

Precision refers to the number of true positives divided by the total positive results predicted by a classifier. Simply put, precision aims to understand what fraction of all positive predictions were actually correct.

$\text{Precision} = \text{True Positives} / (\text{True Positives} + \text{False Positives})$

On the other hand, recall is the number of true positives divided by all the samples that should have been predicted as positive. Recall has the goal to perceive what fraction of actual positive predictions were identified accurately.

$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$



How many selected items are relevant?

Precision = $\frac{\text{true positives}}{\text{true positives} + \text{false positives}}$

How many relevant items are selected?

Recall = $\frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$

In addition to robustness, the F-score shows us how precise a model is by letting us know how many correct classifications are made. The F-score ranges between 0 and 1. The higher the F-score, the greater the performance of the model.

$$F1 = 2 * \frac{1}{\frac{1}{precision} + \frac{1}{recall}}$$

ROC Curve and AUC

ROC curve

An **ROC curve** (**receiver operating characteristic curve**) is a graph showing the performance of a classification model at all classification thresholds. This curve plots two parameters:

- True Positive Rate
- False Positive Rate

True Positive Rate (TPR) is a synonym for recall and is therefore defined as follows:

False Positive Rate (FPR) is defined as follows:

An ROC curve plots TPR vs. FPR at different classification thresholds. Lowering the classification threshold classifies more items as positive, thus increasing both False Positives and True Positives. The following figure shows a typical ROC curve.

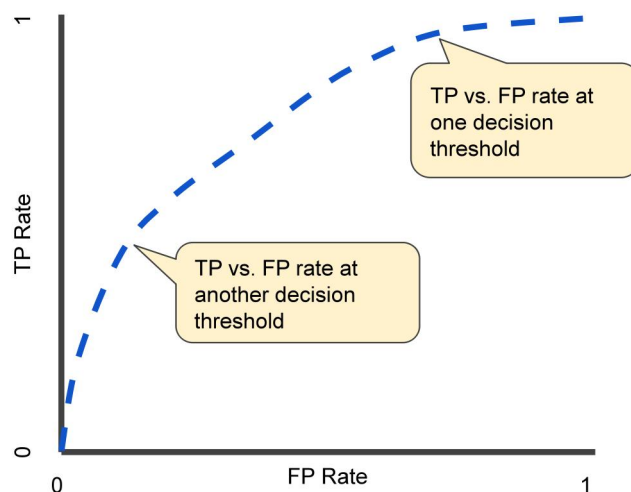


Figure 4. TP vs. FP rate at different classification thresholds.

To compute the points in an ROC curve, we could evaluate a logistic regression model many times with different classification thresholds, but this would be inefficient. Fortunately, there's an efficient, sorting-based algorithm that can provide this information for us, called AUC.

AUC: Area Under the ROC Curve

AUC stands for "Area under the ROC Curve." That is, AUC measures the entire two-dimensional area underneath the entire ROC curve (think integral calculus) from (0,0) to (1,1).

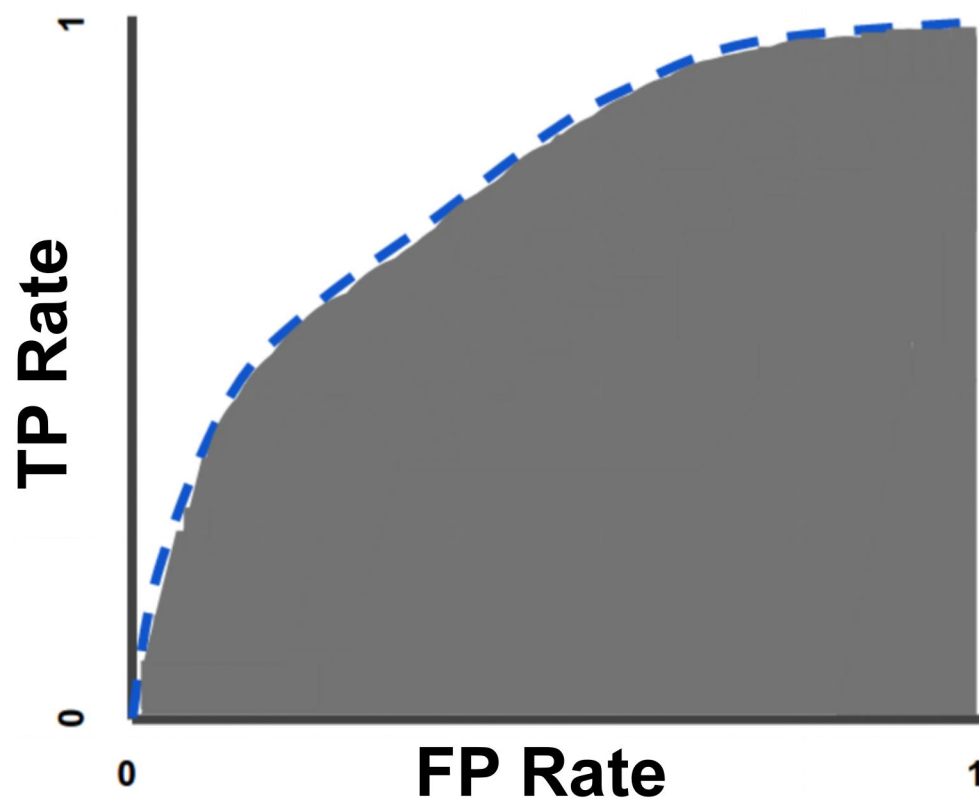


Figure 5. AUC (Area under the ROC Curve).

AUC provides an aggregate measure of performance across all possible classification thresholds. One way of interpreting AUC is as the probability that the model ranks a random positive example more highly than a random negative example. For example, given the following examples, which are arranged from left to right in ascending order of logistic regression predictions:

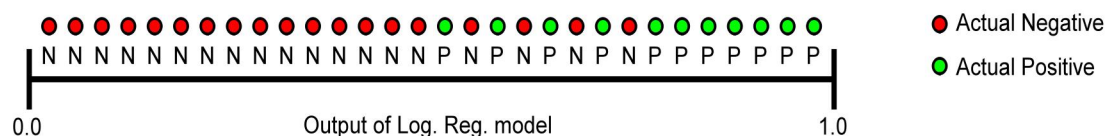


Figure 6. Predictions ranked in ascending order of logistic regression score.

AUC represents the probability that a random positive (green) example is positioned to the right of a random negative (red) example.

AUC ranges in value from 0 to 1. A model whose predictions are 100% wrong has an AUC of 0.0; one whose predictions are 100% correct has an AUC of 1.0.

Metrics for regression problems

Classification models deal with discrete data. The already covered metrics are ideal for classification tasks since they are concerned with whether a prediction is correct. There is no in-between.

Regression models, on the other hand, deal with continuous data. Predictions are in a continuous range. This is the distinction between the metrics for classification and regression problems.

We'll look at a couple of regression metrics.

Mean absolute error

The mean absolute error represents the average of the absolute difference between the original and predicted values.

Mean absolute error provides the estimate of how far off the actual output the predictions were. However, since it's an absolute value, it does not indicate the direction of the error.

Mean absolute error is given by:

$$MeanAbsoluteError = \frac{1}{N} \sum_{j=1}^N |y_j - \hat{y}_j|$$

Mean squared error

The mean squared error is quite similar to the mean absolute error. However, as described by this article, mean squared error uses the average of the square of the difference between original and predicted values. Since this involves the squaring of the errors, larger errors are very notable.

Mean squared error is given by:

$$MeanSquaredError = \frac{1}{N} \sum_{j=1}^N (y_j - \hat{y}_j)^2$$

Root mean squared error

The root mean squared error (RMSE), as defined in this post, computes the idealness of fit by calculating the square root of the average of squared differences between the predicted and actual values. It's a measure of the average error magnitude.

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Improving Performance of a Model

Machine learning development would be not difficult for ML engineers, but ensuring its performance is important to get accurate and most reliable results. Though, there are various methods you can improve your machine learning model performance.

Basically developed on python, machine learning models need to develop while considering the various factors that affect its performance. But right here we brought the list of most important parameters that you can consider while developing the ML model.

Ways to Improve Performance of ML Models

Choosing the Right Algorithms

Algorithms are the key factor used to train the ML models. The data feed into this that helps the model to learn from and predict with accurate results. Hence, choosing the right algorithm is important to ensure the performance of your machine learning model.

Linear Regression, Logistic Regression, Decision Tree, SVM, Naive Bayes, kNN, K-Means, Random Forest and Dimensionality Reduction Algorithms and Gradient Boosting are the leading ML algorithms you can choose as per your ML model compatibility.

Use the Right Quantity of Data

The next important factor you can consider while developing a machine learning model is choosing the right quantity of data sets. And there are multirole factors and for deep learning-based ML models, a huge quantity of datasets is required for algorithms.

Depending on the complexities of problem and learning algorithms, model skill, data size evaluation and use of statistical heuristic rule are the leading factors determine the quantity and types of training data sets that help in improving the performance of the model.

Quality of Training Data Sets

Just like quantity, the quality of machine learning training data set is another key factor, you need to keep in mind while developing an ML model. If the quality of

machine learning training data sets is not good or accurate your model will never give accurate results, affecting the overall performance of the model not suitable to use in real-life.

Actually, there are different methods to measure the quality of the training data set. Standard quality-assurance methods and detailed for in-depth quality assessment are the leading two popular methods you can use to ensure the quality of data sets. Quality of data is important to get unbiased decisions from the ML models, so you need to make sure to use the right quality of training data sets to improve the performance of your ML model.

Supervised or Unsupervised ML

Moreover, the above-discussed ML algorithms, the performance of such AI-based models are affected by methods or process of machine learning. And supervised, unsupervised and reinforcement learning are the algorithm consist of a target/outcome variable (or dependent variable) which is to be predicted from a given set of predictors (independent variables).

In unsupervised machine learning, a model is given any target or outcome variable to predict/estimate. And, it is used for clustering population in different groups, which is widely used for segmenting customers in different groups for specific intervention. For supervised ML, labeled or annotated data is required, while for unsupervised ML the approach is different.

Similarly, reinforcement Learning is another important algorithm, used to train the model to make specific decisions. In this training process, the machine learns from previous experiences and tries to store the best suitable knowledge for the right predictions.

Model Validation and Testing

Building a machine learning model is not enough to get the right predictions, as you have to check the accuracy and need to validate the same to ensure get the precise results. And validating the model will improve the performance of the ML model.

Actually, there are various types of validation techniques you can follow but you need to make sure choose the best one that is suitable for your ML model validation and help you to improve the overall performance of your ML model and predict in an unbiased manner. Similarly, testing of the model is also important to ensure its accuracy and performance.

Add more data samples

Data tells a story only if you have enough of it. Every data sample provides some input and perspective to your data's overall story is trying to tell you. Perhaps the easiest and most straightforward way to improve your model's performance and increase its accuracy is to add more data samples to the training data.

Doing so will add more details to your data and finetune your model resulting in a more accurate performance. Remember after all, the more information you give your model, the more it will learn and the more cases it will be able to identify correctly.

Look at the problem differently

Sometimes adding more data couldn't be the answer to your model inaccuracy problem. You're providing your model with a good technique and the correct dataset. But you're not getting the results you hope for; why?

Maybe you're just asking the wrong questions or trying to hear the wrong story. Looking at the problem from a new perspective can add valuable information to your model and help you uncover hidden relationships between the story variables. Asking different questions may lead to better results and, eventually, better accuracy.

Add some context to your data.

Context is important in any situation, and training a machine learning model is no different. Sometimes, one point of data can't tell a story, so you need to add more context for any algorithm we intend to apply to this data to have a good performance.

More context can always lead to a better understanding of the problem and, eventually, better performance of the model. Imagine I tell you I am selling a car, a BMW. That alone doesn't give you much information about the car. But, if I add the color, model and distance traveled, then you'll start to have a better picture of the car and its possible value.

Finetune your hyperparameter

Training a machine learning model is a skill that you can only hone with practice. Yes, there are rules you can follow to train your model, but these rules don't give you the answer you're seeking, only the way to reach that answer.

However, to get the answer, you will need to do some trial and error until you reach your answer. When I first started learning the different machine learning algorithms, such as the K-means, I was lost on choosing the best number of clusters to reach the optimal results. The way to optimize the results is to tune its hyper-parameters. So, tuning the parameters of the algorithm will always lead to better accuracy.

Train your model using cross-validation

In machine learning, cross-validation is a technique used to enhance the model training process by dividing the overall training set into smaller chunks and then use each chunk to train the model.

Using this approach, we can enhance the algorithm's training process but train it using the different chunks and averaging over the result. Cross-validation is used to optimize the model's performance. This approach is very popular because it's so simple and easy to implement.

Experiment with a different algorithm.

What if you tried all the approaches we talked about so far and your model still results in a low or average accuracy? What then?

Sometimes we choose an algorithm to implement that doesn't really apply to the data we have, and so we don't get the results we expect. Changing the algorithm, you're using to implement your solution. Trying out different algorithms will lead you to uncover more details about your data and the story it's trying to tell.

Feature Engineering

Feature engineering refers to the process of using domain knowledge to select and transform the most relevant variables from raw data when creating a predictive model using machine learning or statistical modeling. The goal of feature engineering and selection is to improve the performance of machine learning (ML) algorithms.

What is Feature Engineering?

The feature engineering pipeline is the preprocessing steps that transform raw data into features that can be used in machine learning algorithms, such as predictive models. Predictive models consist of an outcome variable and predictor variables, and it is during the feature engineering process that the most useful predictor variables are created and selected for the predictive model. Automated feature engineering has been available in some machine learning software since 2016. Feature engineering in ML consists of four main steps: Feature Creation, Transformations, Feature Extraction, and Feature Selection.

Feature engineering consists of creation, transformation, extraction, and selection of features, also known as variables, that are most conducive to creating an accurate ML algorithm. These processes entail:

- **Feature Creation:** Creating features involves identifying the variables that will be most useful in the predictive model. This is a subjective process that requires human intervention and creativity. Existing features are mixed via addition, subtraction, multiplication, and ratio to create new derived features that have greater predictive power.
- **Transformations:** Transformation involves manipulating the predictor variables to improve model performance; e.g. ensuring the model is flexible in the variety of data it can ingest; ensuring variables are on the same scale, making the model easier to understand; improving accuracy; and avoiding computational errors by ensuring all features are within an acceptable range for the model.
- **Feature Extraction:** Feature extraction is the automatic creation of new variables by extracting them from raw data. The purpose of this step is to automatically reduce the volume of data into a more manageable set for modeling. Some feature extraction methods include cluster analysis, text analytics, edge detection algorithms, and principal components analysis.

- **Feature Selection:** Feature selection algorithms essentially analyze, judge, and rank various features to determine which features are irrelevant and should be removed, which features are redundant and should be removed, and which features are most useful for the model and should be prioritized.

Steps in Feature Engineering

The art of feature engineering may vary among data scientists, however steps for how to perform feature engineering for most machine learning algorithms include the following:

- **Data Preparation:** This preprocessing step involves the manipulation and consolidation of raw data from different sources into a standardized format so that it can be used in a model. Data preparation may entail data augmentation, cleaning, delivery, fusion, ingestion, and/or loading.
- **Exploratory Analysis:** This step is used to identify and summarize the main characteristics in a data set through data analysis and investigation. Data science experts use data visualizations to better understand how best to manipulate data sources, to determine which statistical techniques are most appropriate for data analysis, and for choosing the right features for a model.
- **Benchmark:** Benchmarking is setting a baseline standard for accuracy to which all variables are compared. This is done to reduce the rate of error and improve a model's predictability. Experimentation, testing and optimizing metrics for benchmarking is performed by data scientists with domain expertise and business users.

Examples of Feature Engineering

Feature engineering determines the success or failure of a predictive model, and determines how comprehensible the model will be to humans. Advanced feature engineering is at the heart of the Titanic Competition, a popular feature engineering example developed by Kaggle Fundamentals, an online community of data scientists and subsidiary of Google LLC. This project challenges competitors to predict which passengers survived the sinking of the Titanic.

Each Kaggle competition provides a training data set to train the predictive model, and a testing data set to work with. The Titanic Competition also provides information about passengers onboard the Titanic.

Feature Transformation

What is Feature Transformation?

1. It is a technique by which we can boost our model performance. Feature transformation is a mathematical transformation in which we apply a mathematical formula to a particular column(feature) and transform the values which are useful for our further analysis.

2. It is also known as **Feature Engineering**, which is creating new features from existing features that may help in improving the model performance.
3. It refers to the family of algorithms that create new features using the existing features. These new features may not have the same interpretation as the original features, but they may have more explanatory power in a different space rather than in the original space.
4. This can also be used for **Feature Reduction**. It can be done in many ways, by linear combinations of original features or by using non-linear functions.
5. It helps machine learning algorithms to converge faster.

Why These Transformations?

1. Some Machine Learning models, like **Linear and Logistic regression**, assume that the variables follow a normal distribution. More likely, variables in real datasets will follow a skewed distribution.
2. By applying some transformations to these skewed variables, we can map this skewed distribution to a normal distribution so, this can increase the performance of our models.

Goal of Feature Transformations

As we know that **Normal Distribution** is a very important distribution in **Statistics**, which is key to many statisticians for solving problems in statistics. Usually, the data distribution in Nature follows a Normal distribution (**examples like – age, income, height, weight, etc.,**). But the features in the real-life data are not normally distributed, however it is the best approximation when we are not aware of the underlying distribution pattern.

Transformations present in scikit-learn

Sklearn has three Transformations-

1. Function Transformation
2. Power Transformation
3. Quantile transformation

Function Transformations

LOG TRANSFORMATION:

- Generally, these transformations make our data close to a normal distribution but are not able to exactly abide by a normal distribution.
- This transformation is not applied to those features which have negative values.

- This transformation is mostly applied to **right-skewed data**.
- Convert data from additive Scale to multiplicative scale i.e, **linearly distributed data**.

RECIPROCAL TRANSFORMATION

- This transformation is not defined for zero.
- It is a powerful transformation with a **radical effect**.
- This transformation reverses the order among values of the same sign, so large values become smaller and vice-versa.

SQUARE TRANSFORMATION

- This transformation mostly applies to **left-skewed data**.

SQUARE ROOT TRANSFORMATION:

- This transformation is defined only for **positive numbers**.
- This transformation is weaker than Log Transformation.
- This can be used for reducing the **skewness of right-skewed data**.

Power Transformations

- Used when the desired output is more “**Gaussian**” like.
- Currently has ‘**Box-Cox**’ and ‘**Yeo-Johnson**’ transforms.
- **Box-cox** requires the input data to be strictly positive(not even zero is acceptable).
- for features that have zeroes or negative values, **Yeo-Johnson** comes to the rescue.

BOX-COX TRANSFORMATION: Sqrt/sqr/log are the special cases of this transformation.

YEO-JOHNSON TRANSFORMATION: It is a variation of the **Box-Cox** transform.

Quantile transformation

Transform features using quantiles information.

This method transforms the features to follow a uniform or a normal distribution. Therefore, for a given feature, this transformation tends to spread out the most frequent values. It also reduces the impact of (marginal) outliers: this is therefore a robust preprocessing scheme.

The transformation is applied on each feature independently. First an estimate of the cumulative distribution function of a feature is used to map the original values to a uniform distribution. The obtained values are then mapped to the desired output distribution using the associated quantile function. Features values of new/unseen data that fall below or above the fitted range will be mapped to the bounds of the output distribution. Note that this transform is non-linear. It may distort linear correlations between variables measured at the same scale but renders variables measured at different scales more directly comparable.

Feature Subset Selection

Feature Selection is the most critical pre-processing activity in any machine learning process. It intends to select a subset of attributes or features that makes the most meaningful contribution to a machine learning activity. In order to understand it, let us consider a small example i.e. **Predict the weight of students based on the past information about similar students**, which is captured inside a ‘Student Weight’ data set. The data set has 04 features like **Roll Number, Age, Height & Weight**. Roll Number has no effect on the weight of the students, so we eliminate this feature. So now the new data set will be having only 03 features. This subset of the data set is expected to give better results than the full set.

Before proceeding further, we should look at the fact why we have reduced the dimensionality of the above dataset OR what are the issues in **High Dimensional Data**?

High Dimensional refers to the high number of variables or attributes or features present in certain data sets, more so in the domains like DNA analysis, geographic information system (GIS), etc. It may have sometimes hundreds or thousands of dimensions which is not good from the machine learning aspect because it may be a big challenge for any ML algorithm to handle that. On the other hand, a high quantity of computational and a high amount of time will be required. Also, a model built on an extremely high number of features may be very difficult to understand. **For these reasons, it is necessary to take a subset of the features instead of the full set.** So we can deduce that the objectives of feature selection are:

1. Having a faster and more cost-effective (less need for computational resources) learning model
2. Having a better understanding of the underlying model that generates the data.
3. Improving the efficacy of the learning model.

Main Factors Affecting Feature Selection

a. Feature Relevance: In the case of supervised learning, the input data set (which is the training data set), has a class label attached. A model is inducted based on the training data set — so that the inducted model can assign class labels to new, unlabeled data. Each of the predictor variables, ie expected to contribute information to decide the value of the class label. In case of a variable is not contributing any information, it is said to be irrelevant. In case the information contribution for

prediction is very little, the variable is said to be weakly relevant. The remaining variables, which make a significant contribution to the prediction task are said to be strongly relevant variables.

In the case of unsupervised learning, there is no training data set or labelled data. Grouping of similar data instances are done and the similarity of data instances are evaluated based on the value of different variables. Certain variables do not contribute any useful information for deciding the similarity of dissimilar data instances. Hence, those variable makes no significant contribution to the grouping process. These variables are marked as irrelevant variables in the context of the unsupervised machine learning task.

We can understand the concept by taking a real-world example: At the start of the article, we took a random dataset of the student. In that, Roll Number doesn't contribute any significant information in predicting what the Weight of a student would be. Similarly, if we are trying to group together students with similar academic capabilities, *Roll No* can really not contribute any information. So, in the context of grouping students with similar academic merit, the variable *Roll No* is quite irrelevant. Any feature which is irrelevant in the context of a machine learning task is a candidate for rejection when we are selecting a subset of features.

b. Feature Redundancy: A feature may contribute to information that is similar to the information contributed by one or more features. For example, in the Student Data-set, both the features **Age & Height** contribute similar information. This is because, with an increase in age, weight is expected to increase. Similarly, with the increase in Height also weight is expected to increase. So, in context to that problem, Age and Height contribute similar information. In other words, irrespective of whether the feature **Height** is present or not, the learning model will give the same results. In this kind of situation where one feature is similar to another feature, **the feature is said to be potentially redundant** in the context of a machine learning problem.

All features having potential redundancy are candidates for rejection in the final feature subset. Only a few representative features out of a set of potentially redundant features are considered for being a part of the final feature subset. So in short, the main objective of feature selection is to remove all features which are irrelevant and take a representative subset of the features which are potentially redundant. This leads to a meaningful feature subset in the context of a specific learning task.

The measure of feature relevance and redundancy

a. Measures of Feature Relevance: In the case of supervised learning, **mutual information** is considered as a good measure of information contribution of a feature to decide the value of the class label. That is why it is a good indicator of the relevance of a feature with respect to the class variable. The higher the value of mutual information of a feature, the more relevant is that feature. Mutual information can be calculated as follows:

$$MI(C, f) = H(C) + H(f) - H(C, f)$$

Where, **marginal entropy of the class,**
 $H(C) = - \sum_{i=1}^K p(C_i) \log_2 p(C_i)$

Marginal entropy of the feature
 $H(f) = - \sum_c p(f = x) \log_2 p(f = x)$

And **K** = number of classes, **C** = class variable, **f** = feature set that take discrete values. In the case of unsupervised learning, there is no class variable. Hence, feature-to-class mutual information cannot be used to measure the information contribution of the features. In the case of unsupervised learning, the entropy of the set of features without one feature at a time is calculated for all features. Then the features are ranked in descending order of information gain from a feature and the top β percentage (value of beta is a design parameter of the algorithm) of features are selected as relevant features. The entropy of a feature f is calculated using Shannon's formula below:

$$H(f) = - \sum_x p(f = x) \log_2 p(f = x)$$

$\sum_x$ is used only for features that take the discrete values. For continuous features, it should be replaced by discretization performed first to estimate the probabilities $p(f=x)$.

b. Measures of Feature Redundancy: There are multiple measures of similarity of information contribution, the main ones are:

- Correlation-based Measures
- Distance-based Measures
- Other coefficient-based Measure

1. Correlation Based Similarity Measure

Correlation is a measure of linear dependency between two random variables. Pearson's product correlation coefficient is one of the most popular and accepted measures correlation between two random variables. For two random feature variables F_1 and F_2 , the Pearson coefficient is defined as:

$$\sigma = \frac{cov(F_1, F_2)}{\sqrt{var(F_1) \cdot var(F_2)}}$$

$$cov(F_1, F_2) = \sum (F_{1_i} - \overline{F_1}) \cdot (F_{2_i} - \overline{F_2})$$

$$var(F_1) = \sum (F_{1_i} - \overline{F_1})^2, \text{ where } \overline{F_1} = \frac{1}{n} \cdot \sum F_{1_i}$$

$$var(F_2) = \sum (F_{2_i} - \overline{F_2})^2, \text{ where } \overline{F_2} = \frac{1}{n} \cdot \sum F_{2_i}$$

Correlation value ranges between +1 and -1. A correlation of 1 (+/-) indicates perfect correlation. In case the correlation is zero, then the features seem to have no linear relationship. Generally for all feature selection problems, a threshold value is adopted to decide whether two features have adequate similarity or not.

2. Distance-Based Similarity Measure

The most common distance measure is the **Euclidean distance**, which, between two features F_1 and F_2 are calculated as:

$$d(F_1, F_2) = \sqrt{\sum_{i=1}^n (F_{1_i} - F_{2_i})^2}$$

Where the features represent an n-dimensional dataset. Let us consider that the dataset has two features, Subjects (F_1) and marks (F_2) under consideration. The Euclidean distance between the two features will be calculated like this:

Subjects (F_1)	Marks (F_2)	($F_1 - F_2$)	($F_1 - F_2$) ²
2	6	-4	16
3	5.5	-2.5	6.25
6	4	2	4
7	2.5	4.5	20.25
8	3	5	25
6	5.5	0.5	0.25
6	7	-1	1
7	6	1	1
8	6	2	4
9	7	2	4

A more generalized form of the Euclidean distance is the **Minkowski Distance**,

measured as
$$d(F_1, F_2) = \sqrt[r]{\sum_{i=1}^n |F_{1_i} - F_{2_i}|^r}$$

Minkowski distance takes the form of Euclidean distance (also called **L₂ norm**) where $r = 2$. At $r=1$, it takes the form of **Manhattan** distance (also called **L₁ norm**):
$$d(F_1, F_2) = \sum_{i=1}^n |F_{1_i} - F_{2_i}|$$

3. Other Similarity Measures

Jaccard index/coefficient is used as a measure of dissimilarity between two features is complementary of Jaccard Index. For two features having binary values, Jaccard

Index is measured as:
$$J = \frac{n_{11}}{n_{01} + n_{10} + n_{11}}$$

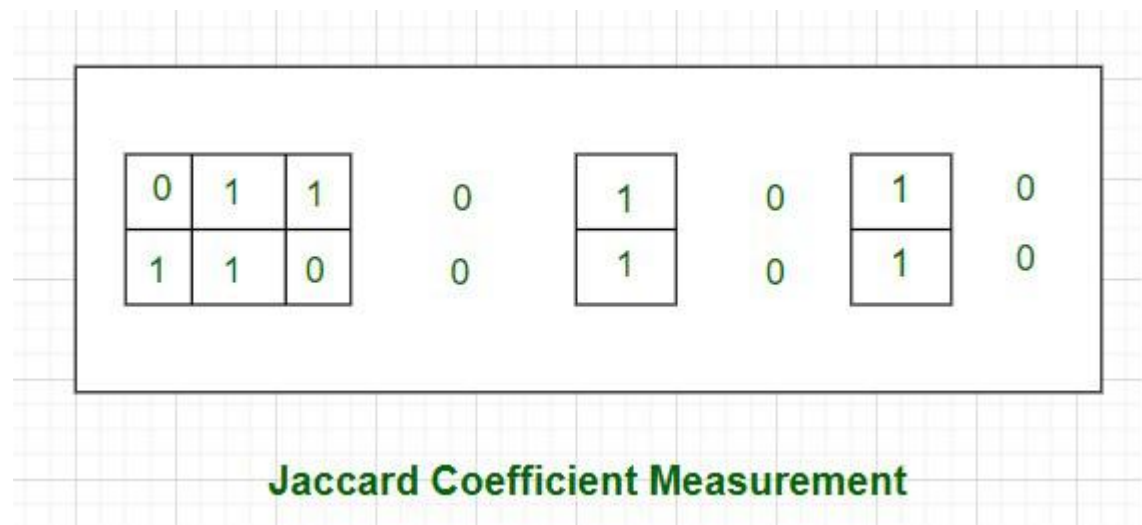
Where n_{11} = number of cases when both the feature have value 1,

n_{01} = number of cases where the feature 1 has value 0 and feature 2 has value 1,

n_{10} = the number of cases where feature 1 has value 1 and feature 2 has value 0.

Jaccard distance: $d_j = (1 - j)$

Let us take an example to understand it better. Consider two features, F_1 and F_2 having values (0, 1, 1, 0, 1, 0, 1, 0) and (1, 1, 0, 0, 1, 0, 0, 0).



As shown in the above picture, the cases where both the values are 0 have been left out without border- as an indication of the fact that they will be excluded in the calculation of the Jaccard coefficient.

Jaccard coefficient of F_1 and F_2 , $J = \frac{n_{11}}{n_{01} + n_{10} + n_{11}} = \frac{2}{1+2+2} = 0.4$

Therefore, Jaccard Distance between those two features is $d_j = (1 - 0.4) = 0.6$

Note: One more measure of similarity using similarity coefficient calculation is **Cosine Similarity**. For the sake of understanding, let us take an example of the **text classification problem**. The text needs to be first transformed into features with a word token being a feature and the number of times the word occurs in a document comes as a value in each row. There are thousands of features in such a text dataset. However, the data set is sparse in nature as only a few words do appear in a document and hence in a row of the data set. So each row has very few non-zero values. However, the non-zero values can be anything integer value as the same word may occur any number of times. Also, considering the sparsity of the dataset, the 0-0 matches need to be ignored. **Cosine similarity** which is one of the most popular measures in text classification is calculated as:

$$\cos(x, y) = \frac{x \cdot y}{||x|| \cdot ||y||}$$

Where, $x \cdot y$ is the vector dot product of x and $y = \sum_{i=1}^n x_i y_i$

$$||x|| = \sqrt{\sum_{i=1}^n x_i^2} \text{ and } ||y|| = \sqrt{\sum_{i=1}^n y_i^2}$$

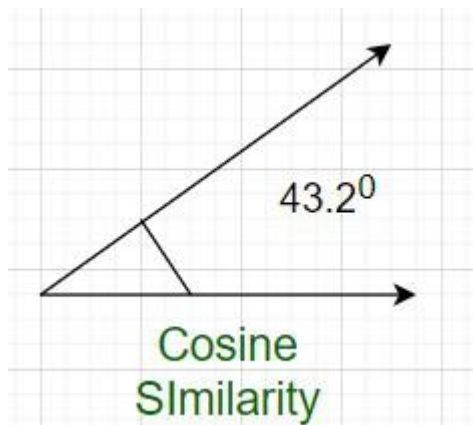
So let's calculate the cosine similarity of x and y , where $x = (2, 4, 0, 0, 2, 1, 3, 0, 0)$ and $y = (2, 1, 0, 0, 3, 2, 1, 0, 1)$. In this case, dot product of x and y will be $x \cdot y = 2*2 + 4*1 + 0*0 + 0*0 + 2*3 + 1*2 + 3*1 + 0*0 + 0*1 = 19$.

$$||x|| = \sqrt{2^2 + 4^2 + 0^2 + 0^2 + 2^2 + 1^2 + 3^2 + 0^2 + 0^2} = \sqrt{34} = 5.83$$

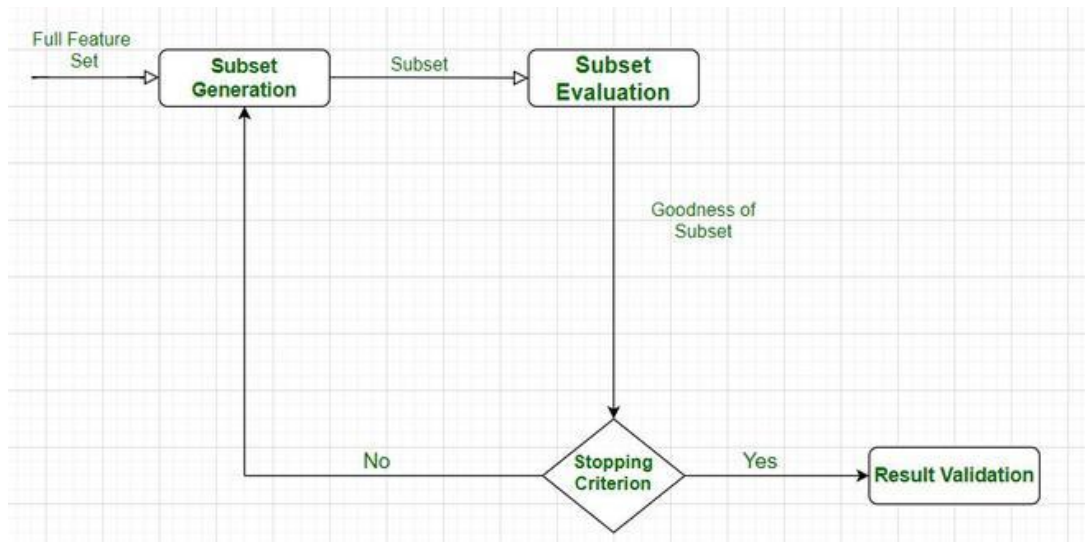
$$||y|| = \sqrt{2^2 + 1^2 + 0^2 + 0^2 + 3^2 + 2^2 + 1^2 + 0^2 + 1^2} = \sqrt{20} = 4.47$$

$$\therefore \cos(x, y) = \frac{19}{5.83*4.47} = 0.729$$

Cosine Similarity measures the angle between x and y vectors. Hence, if cosine similarity has a value of 1, the angles between x and y is 0 degrees which means x and y are the same except for the magnitude. If the cosine similarity is 0, the angle between x and y is 90° . Hence, they do not share any similarity. In the case of the above example, the angle comes out to be 43.2° .



Even after all these steps, there are some few more steps. You can understand it by the following flowchart:



Feature Selection Process

After the successful completion of this cycle, we get the desired features, and we have finally tested them also.

Feature Selection Models

Feature selection models are of two types:

1. **Supervised Models:** Supervised feature selection refers to the method which uses the output label class for feature selection. They use the target variables to identify the variables which can increase the efficiency of the model
2. **Unsupervised Models:** Unsupervised feature selection refers to the method which does not need the output label class for feature selection. We use them for unlabelled data.

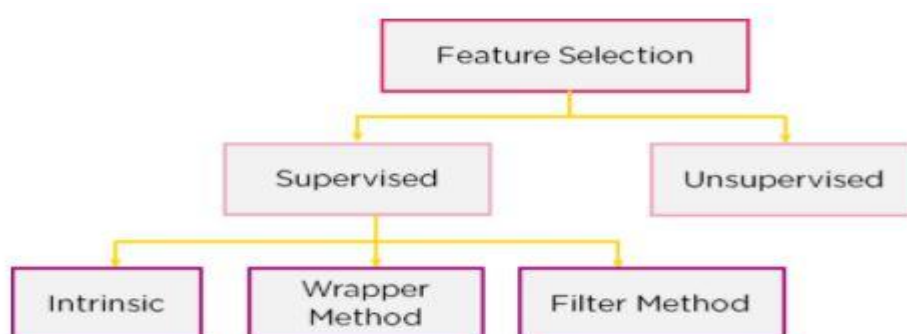


Figure 4: Feature Selection Models

We can further divide the supervised models into three :

1. Filter Method: In this method, features are dropped based on their relation to the output, or how they are **correlating** to the output. We use correlation to check if the features are positively or negatively correlated to the output labels and drop features accordingly. Eg: Information Gain, Chi-Square Test, Fisher's Score, etc.

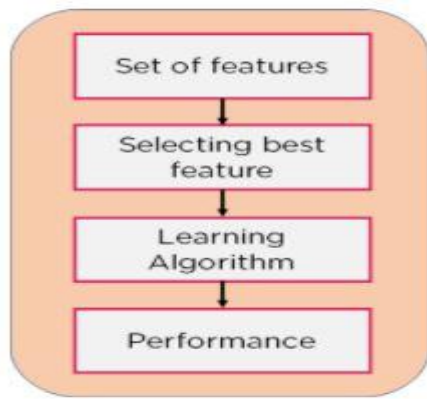


Figure 5: Filter Method flowchart

2. Wrapper Method: We split our data into subsets and train a model using this. Based on the output of the model, we add and subtract features and train the model again. It forms the subsets using a greedy approach and evaluates the accuracy of all the possible combinations of features. Eg: Forward Selection, Backwards Elimination, etc.

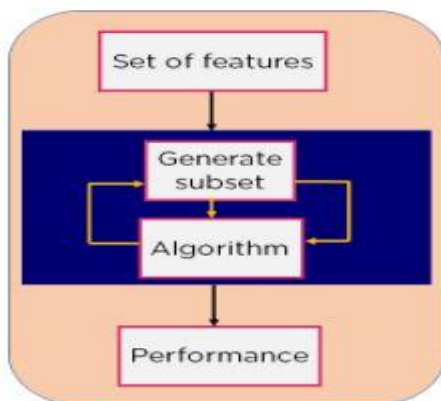


Figure 6: Wrapper Method Flowchart

3. Intrinsic Method: This method combines the qualities of both the Filter and Wrapper method to create the best subset.

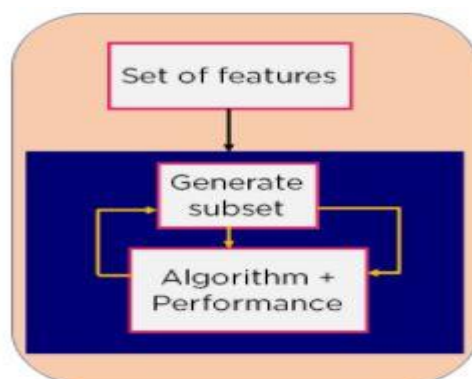


Figure 7: Intrinsic Model Flowchart

This method takes care of the machine training iterative process while maintaining the computation cost to be minimum. Eg: Lasso and Ridge Regression.

How to Choose a Feature Selection Model?

How do we know which feature selection model will work out for our model? The process is relatively simple, with the model depending on the types of input and output variables.

Variables are of two main types:

- Numerical Variables: Which include integers, float, and numbers.
- Categorical Variables: Which include labels, strings, boolean variables, etc.

Based on whether we have numerical or categorical variables as inputs and outputs, we can choose our feature selection model as follows:

Input Variable	Output Variable	Feature Selection Model
Numerical	Numerical	• Pearson's correlation coefficient • Spearman's rank coefficient
Numerical	Categorical	• ANOVA correlation coefficient (linear). • Kendall's rank coefficient (nonlinear).
Categorical	Numerical	• Kendall's rank coefficient (linear). • ANOVA correlation coefficient (nonlinear).
Categorical	Categorical	• Chi-Squared test (contingency tables). • Mutual Information.

Table 1: Feature Selection Model lookup